

REPORT 05 · FACTORY BENT SIMULATION WITH A BCR ROBOT ARM

Worlds & Product Generator Evolution

From one straight belt to a 12-arm pipeline factory,
from 7 fixed products to 100 + your own

ROS 2 Jazzy · Gazebo Harmonic · OpenCV · PyQt5 · BCR 7-axis arm

Hüseyin Tozluyurt · November 2026

1. Worlds

1.1 The three original worlds

The project started with `factory_conveyor.sdf`, a single straight bent with 7 hand-written products moving at 0.18 m/s. Next came `circular_factory_conveyor.sdf` (one ring) and `dual_circular_factory_conveyor.sdf` (two rings, a placeholder for the arm base, one `pickup_box`). All three publish an overhead camera.

1.2 The dual world becomes the reference cell

- **station / sidecam / ring2**: the products became separate models, so each can be picked. A `side_camera` and a `release_camera` were added.
- **pictures**: a `product_photo_camera` at (0, 11, 1.6) and a small photo studio, away from the belts (`.bak_before_photocam`).
- **style**: a factory hall, epoxy floor, yellow safety fence with a gate and stack lights, hazard frame, walkways, pallet racks, an operator desk and BENT 1 PICK / BENT 2 PLACE signs. All of it is visual only and kept out of the arm's reach. `dual_circular_factory_conveyor_plain.sdf` keeps the old look.
- **bins**: two open bins with collision walls.

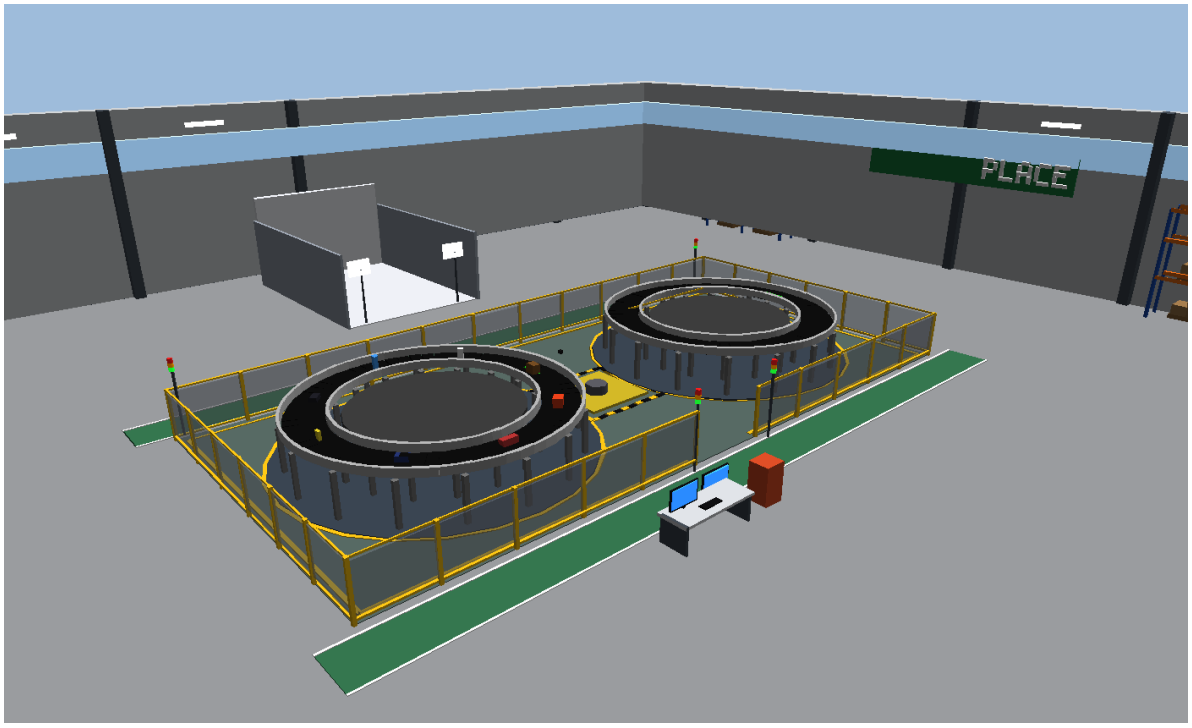
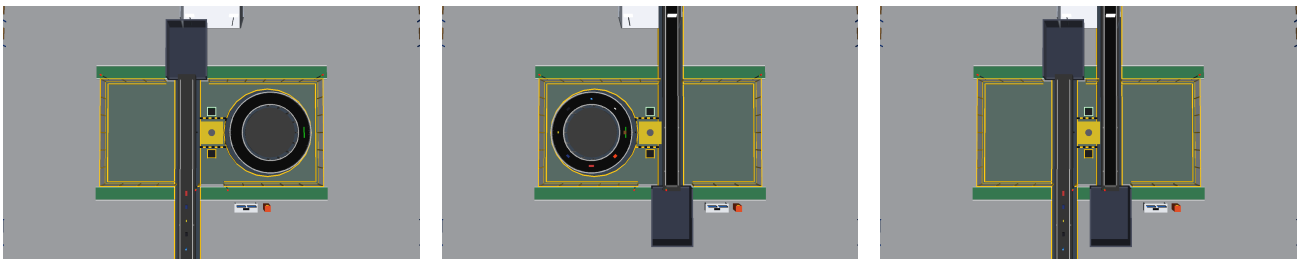


Figure 1. The decorated dual circular world (generator: style_previews/world_decor_generator.py).

1.3 One cell, any belt (worlds, exp)

The other worlds were not edited by hand. They were **generated from the dual world** by `make_worlds.py` and `make_exp_worlds.py`, so the arm, catch station (-2.58, 0), place point (2.22, -0.37), cameras and bins are in exactly the same places. A straight bent 1 runs +y along x = -2.58, and a straight bent 2 runs −y along x = 2.22. Each ends over a collection box, where products fall and stay. `product_catalog.WORLD_LAYOUTS` describes each world's belts. `factory_full.launch.py` reads the `<world name>` from the SDF and sets `FACTORY_WORLD_NAME`, so every node picks the right layout.



exp_straight_circular.sdf exp_circular_straight.sdf exp_straight_straight.sdf

Figure 2. The three experimental worlds (top views).

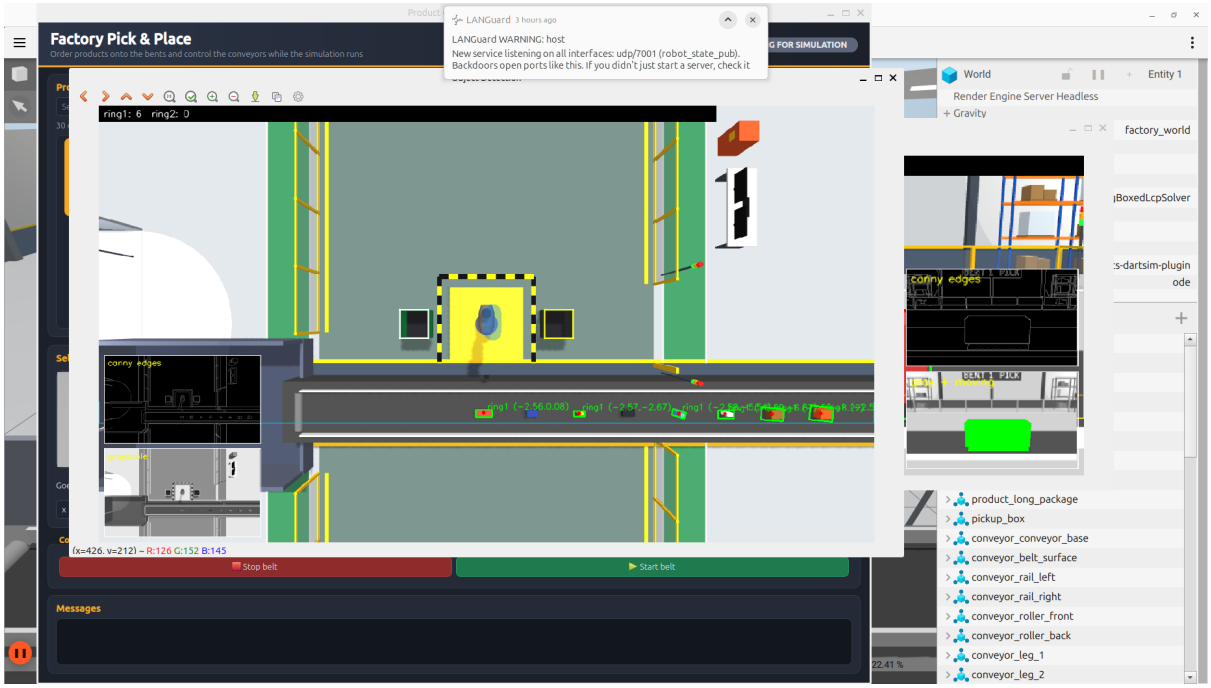


Figure 3. The straight bent with live detection (Oct 31, 06:23).

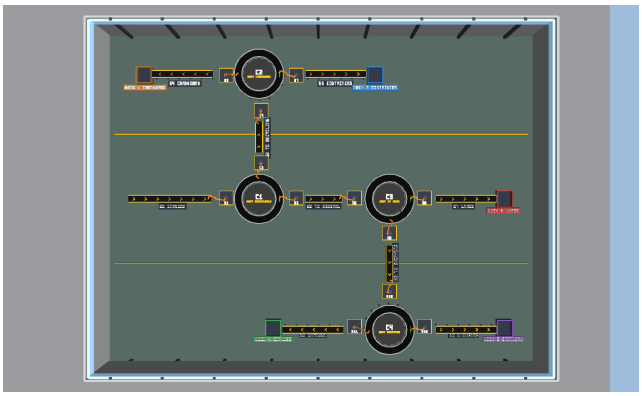
Capacity of one order	at 1.0 m spacing
circular bent (radius 3.42 m)	19 products (39 at 0.5 m)
straight bent (before the station)	14 products

1.4 The pipeline factory

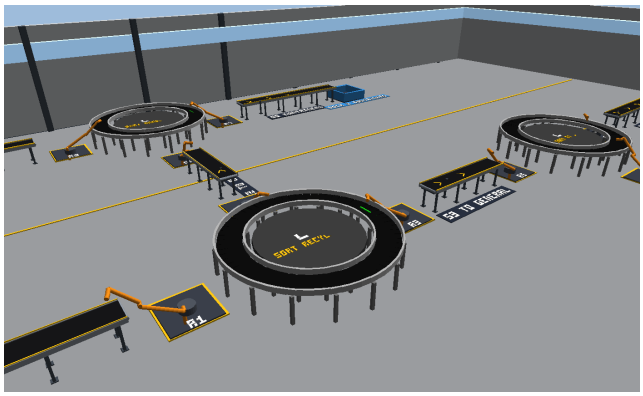
`make_pipeline_world.py` builds a 13-conveyor factory from the table in `pipeline_layout.py`: 4 sorting rings, 9 transport lines, 12 BCR arms (with real meshes through `model://bcr_arm_description`) and 5 docks, each with a label painted on the floor. `make_pipeline_lab_world.py` adds a camera and two bins at each station, plus movable arm links (`pipeline_factory_lab.sdf`).

Arm	From	To	Takes
A1	S1 inbound	C1	all
A2	C1	S2 → recycling	recyclable
A3	C1	S3 → general	the rest
A4	S2	C2	all
A6	C2	S4, dock 1	cardboard
A7	C2	S5, dock 2	the rest
A5	S3	C3	all

Arm	From	To	Takes
A9	C3	S7, dock 3	large
A8	C3	S6 → shipping	the rest
A10	S6	C4	all
A11	C4	S8, dock 4	small
A12	C4	S9, dock 5	the rest



Top view



Perspective

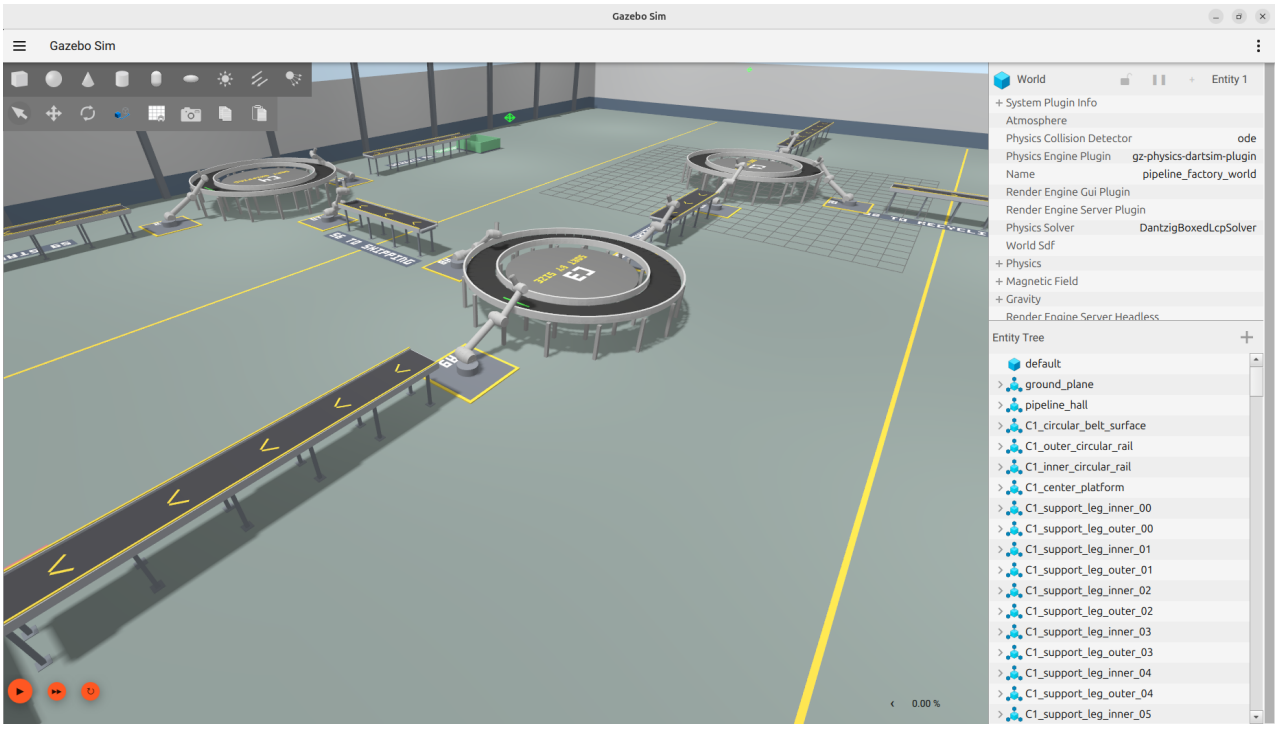


Figure 4. The pipeline factory in Gazebo (Oct 31, 07:24).

2. The product generator

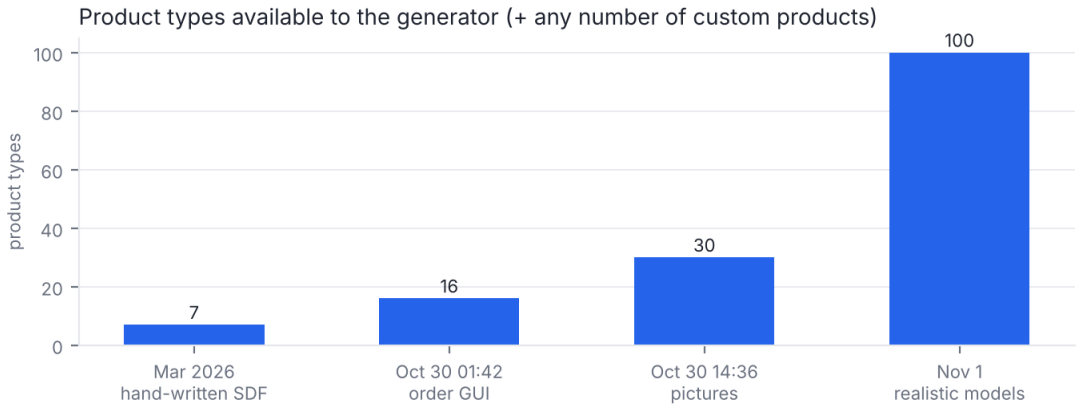


Figure 5. Built-in product types over time.

2.1 Orders instead of fixed products (order phase)

Products used to be written into the SDF. Since Oct 30 they are spawned by the attacher from `product_catalog.py`, at equal arc distances along bent 1, with the first one `start_offset` before the station. Spawned models are named `order_<type>_<n>`, so the type, size and colour can always be looked up from the name. Bent 1 stops while they are created, and then runs again.

```
ros2 topic pub --once /spawn_order std_msgs/msg/String \
  "{data: '{\"bent\": 1, \"items\": [\"paint_can\", \"cereal_box\"],
            \"spacing\": 1.0, \"start_offset\": 1.5, \"clear\": true}'}"
```

2.2 Pictures (pictures phase)

The catalogue grew to 30 types (boxes, cylinders and two balls). The rule for new types: the top must be at least 8 × 8 cm and the height at least 8 cm, so both cameras see it. `capture_product_images` puts each type in front of the photo camera and saves the image for the GUI's picture grid.

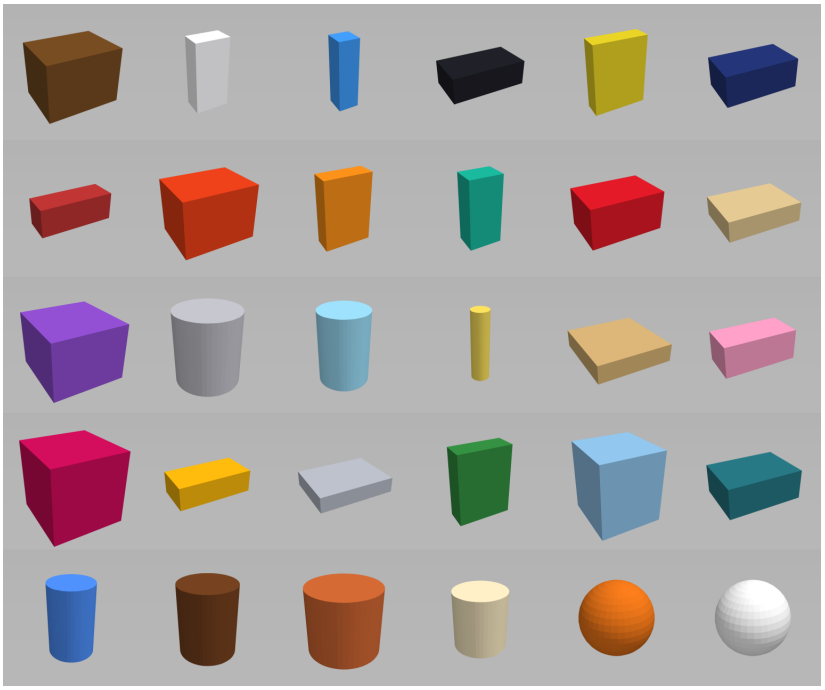
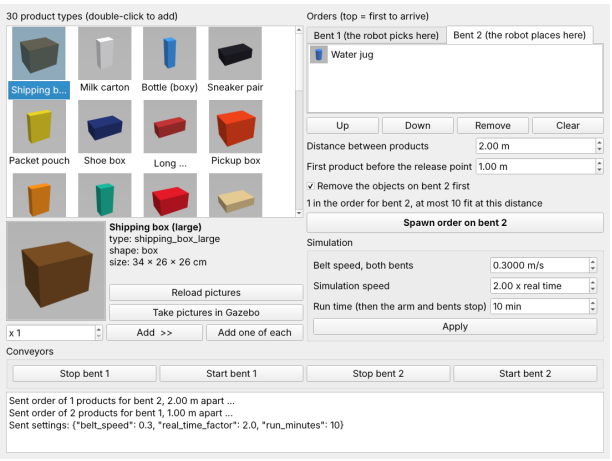


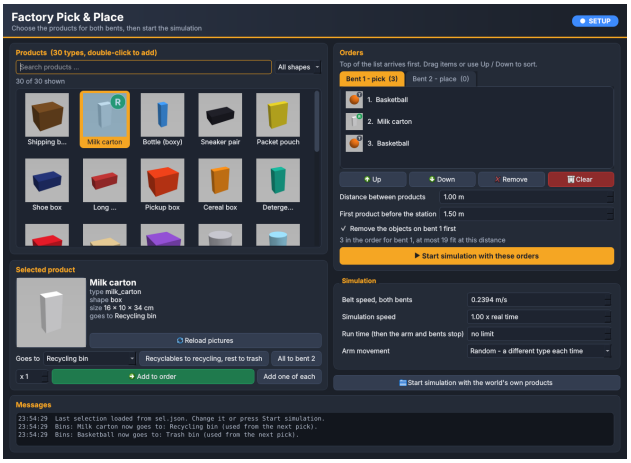
Figure 6. The 30 product pictures of Oct 30.

2.3 The order GUI

It started as two lists with Up/Down buttons. It gained a picture grid, bent 1 and bent 2 tabs, a capacity check, belt speed, simulation speed, run time and arm movement controls, a dark theme with a status light, search and shape filters, drag-to-sort, "Goes to" choices with R/T badges, and auto-sort. With `run_factory.sh` it opens first as a start-up selector, and each world remembers its last order.



Oct 30: first picture version



Oct 30 night: dark theme, bins

2.4 Realistic products (Nov 1)

`product_models.py` gives each of the **100 types** (66 boxes, 26 cylinders, 8 spheres) a detailed visual made of primitive parts, such as the gable roof of a milk carton, the neck and cap of a bottle, or the ribbon of a gift box. Printed labels are PNG textures drawn with OpenCV for fictional brands. The collision shape stays the simple catalogue box, cylinder or sphere, so grasping, bins and physics are unchanged. Every part stays inside the catalogue size, and the top keeps the catalogue colour for the detectors.



Figure 7. The 100 realistic product types.

2.5 Your own products

`custom_products.py` lets users add a product from an STL, OBJ or DAE model (scaled to the given size and centred, with its .mtl and textures copied) or from a simple shape with a photo printed on its sides. Products can be exported and imported as .zip files. The size rules are at most 48 × 34 × 42 cm, at least 6 cm high, a top of at least 60 cm², and an aspect ratio of at most 4.5. Custom products join spawning, bins, the OpenCV detectors, the colour classifier and YOLO datasets.

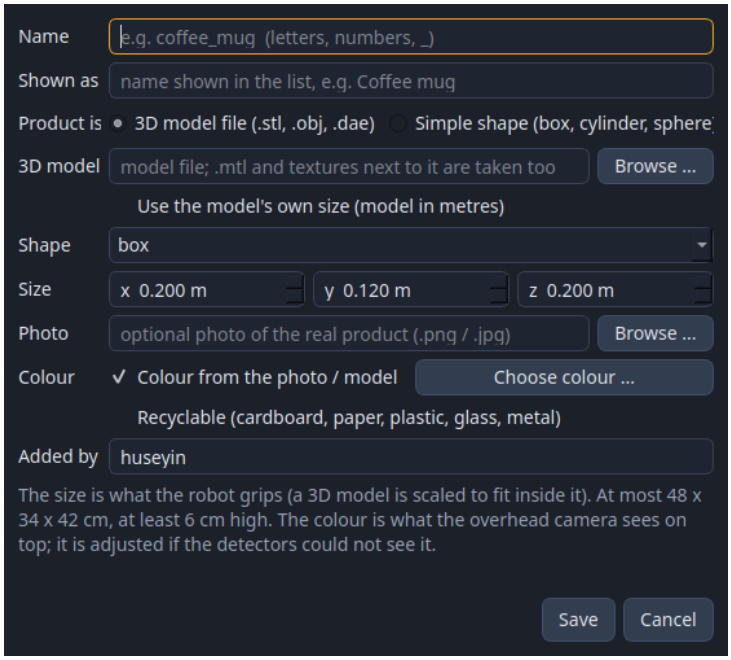


Figure 8. The Add-your-own-product dialog.